SECURITY VULNERABILITY REPORT

Predictable Security-Sensitive Identifiers via mt_rand()

PHPGurukul Beauty Parlour Management System

Finding F-NEW-01

SEVERITY MEDIUM | CVSS 4.0 5.0 | CWE 330 | OWASP A02:2021

Product	PHPGurukul Beauty Parlour Management System
Version	Product family v1.x (DB dump Sep 2024)
Target	http://localhost/bpms/ (authorized localhost XAMPP lab)
Assessment date	2026-10-30
Report version	1.0
Status	DRAFT - for human review / coordinated disclosure
Evidence (sealed)	runs/bpms/localhost/1793369337 SEAL 780fe797...

Scope & authorization

All testing was performed on an authorized local lab. No production systems or third-party data were touched. No CVE is claimed or assigned. PoCs are non-destructive; the database was restored to baseline after testing.

Table of Contents

1 Executive Summary 3

2 Vulnerability Metadata 3

3 Attack Flow 3

4 Technical Description 4

5 Source Code Evidence 4

6 Proof of Concept (reproduced, non-destructive) 5

7 HTTP Request / Response 6

8 Runtime Evidence 6

9 Screenshots 6

10 Impact 6

11 CVE Comparison & Novelty Assessment 7

12 Remediation 7

13 Verification Checklist 7

1 Executive Summary

BPMS issues each customer an appointment number and each invoice a billing number, and uses those numbers as the key to look a record up for display. The numbers are produced with PHP's mt_rand() - a fast but non-cryptographic random function - and are only nine digits long.

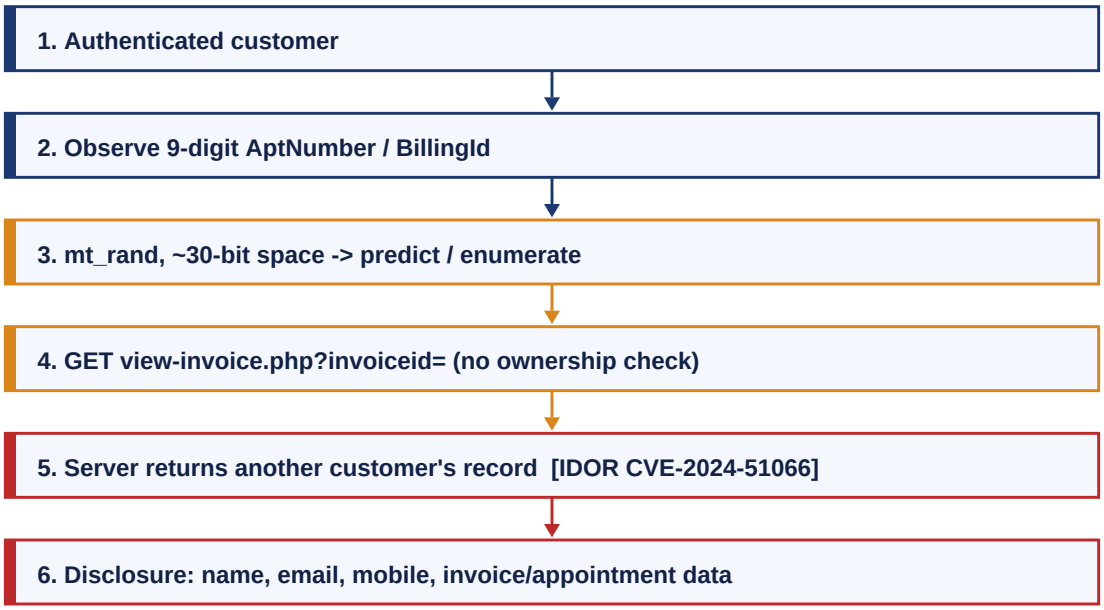
Because the pages that display a record by its number do not verify that the record belongs to the requester (a separate access-control flaw; the appointment case is public as CVE-2024-51066), the number is effectively the only barrier between one customer and another customer's data. A predictable generator over a small range is therefore a distinct weakness that lowers the effort to guess or enumerate another customer's appointment/invoice and read their personal information.

> **Bottom line**
> Medium on its own (CWE-330, insecure randomness). Materially easier to exploit when combined with the missing ownership check (CVE-2024-51066) and the absence of rate limiting. Appears unreported for this product as of 2026-10-30.

2 Vulnerability Metadata

Field	Value
Finding ID	F-NEW-01
Severity	Medium
CVSS 4.0	CVSS:4.0/AV:N/AC:H/AT:P/PR:L/UI:N/VC:L/VI:N/VA:N (approx 5.0)
CWE	CWE-330 Use of Insufficiently Random Values (rel. CWE-340)
OWASP 2021	A02 Cryptographic Failures (amplified by A01 Broken Access Control)
Affected files	book-appointment.php:16 ; admin/add-customer-services.php:12
Exposed via	view-invoice.php?invoiceid= ; appointment-detail.php?aptnumber=
Authentication	Low-priv customer (appt-detail IDOR reported unauth: CVE-2024-51066)
Impact	Confidentiality of other customers' PII; no integrity/availability impact
Reproduction	Confirmed (source + runtime); PRNG state recovery not demonstrated

Attack flow



4 Technical Description

4.1 Root cause

Vulnerable identifier generation (highlighted)

```
// book-appointment.php:16
$aptnumber = mt_rand(100000000, 999999999);

// admin/add-customer-services.php:12
$invoiceid = mt_rand(100000000, 999999999);
```

4.2 Why mt_rand() is insecure here

mt_rand() implements the Mersenne Twister, a deterministic PRNG auto-seeded once per request. It is built for speed and statistical quality, not unpredictability; given enough observed outputs its state can be reconstructed, and PHP recommends random_int()/random_bytes() for security-sensitive values. The output is also constrained to [100000000, 999999999] - about 10^9 ~ 2^30 possibilities - a small space for a value that gates access to personal data.

4.3 Why it matters in BPMS

```
// view-invoice.php:78-82  (intval blocks SQLi, but NO ownership check)
$invid = intval($_GET['invoiceid']);
$ret = mysqli_query($con,
  "select ... from tblinvoice
     join tbluser on tbluser.ID = tblinvoice.Userid
    where tblinvoice.BillingId = '$invid'");
```

With ownership unchecked, the identifier is the only barrier - so its predictability and small range are a security weakness in their own right (distinct CWE and fix from the IDOR).

5 Source Code Evidence

File	Line	Issue
book-appointment.php	16	Insecure PRNG for AptNumber

File	Line	Issue
admin/add-customer-services.php	12	Insecure PRNG for BillingId
view-invoice.php	78-82	No ownership check (amplifier)
appointment-detail.php	77-78	No ownership check - CVE-2024-51066

6 Proof of Concept (reproduced, non-destructive)

Run on the authorized local lab, 2026-10-30. Session cookies redacted. Two test bookings were created then deleted (DB restored to baseline).

Step 1 - Authenticate as a customer

```
curl -s -c cust.jar -b cust.jar -o /dev/null -w 'login HTTP %{http_code}\n' \
  --data 'emailcont=johndoe@gmail.com&password=Test@123&login=' \
  http://localhost/bpms/login.php
```

output

```
login HTTP 302
```

Step 2 - Create two appointments

```
curl -s -b cust.jar -o /dev/null -w 'book#1 HTTP %{http_code}\n' \
  --data 'adate=2030-03-03&atime=11:00&message=PoC-INDIV-1&submit=' \
  http://localhost/bpms/book-appointment.php
# (repeat for booking #2)
```

output

```
book#1 HTTP 200
book#2 HTTP 200
```

Step 3 - Read the generated identifiers (DB truth)

```
/opt/lampp/bin/mysql -uroot bpmsdb \
  -e "SELECT ID,AptNumber,AptDate,Message FROM tblbook WHERE Message LIKE 'PoC-INDIV%';"
```

output

```
ID   AptNumber   AptDate      Message
14   593951775   2030-03-03   PoC-INDIV-1
15   344823593   2030-04-04   PoC-INDIV-2
```

Step 4 - Identifiers are exposed to the user

```
curl -s -b cust.jar http://localhost/bpms/booking-history.php | grep -oE '[0-9]{9}' | sort -u
```

output

```
226906690
344823593
593951775
789654123
```

Observation
Each AptNumber is a 9-digit value in mt_rand's range; the existing invoice BillingId is 193862954 (same generator). booking-history.php renders these values to the user, revealing the identifier format and sample values.

Step 5 - Cleanup

```
/opt/lampp/bin/mysql -uroot bpmsdb -e "DELETE FROM tblbook WHERE Message LIKE 'PoC-INDIV%';"
# tblbook row count returned to 2 (baseline). Verified.
```

7 HTTP Request / Response

request

```
POST /bpms/book-appointment.php HTTP/1.1
Host: localhost
Content-Type: application/x-www-form-urlencoded
Cookie: PHPSESSID=<REDACTED_CUSTOMER_SESSION>

adate=2030-03-03&atime=11:00&message=PoC-INDIV-1&submit=
```

response (headers captured; body is a client-side redirect)

```
HTTP/1.1 200 OK
Date: Fri, 30 Oct 2026 14:08:57 GMT
Server: Apache/2.4.58 (Unix) OpenSSL/1.1.1w PHP/8.2.12 mod_perl/2.0.12 Perl/v5.34.1
Content-Type: text/html; charset=UTF-8

<script>window.location.href='thank-you.php'</script>
<!-- AptNumber for this request = 593951775 (9 digits, mt_rand range) -->
```

8 Runtime Evidence

identifiers observed this run (DB)

```
AptNumber  593951775     (booking ID 14)
AptNumber  344823593     (booking ID 15)
BillingId  193862954     (existing invoice)
```

All values lie within mt_rand(100000000, 999999999). Saved: evidence/generated_aptnumbers.txt, evidence/invoice_billingids.txt, evidence/booking_history.html.

9 Screenshots

No screenshots captured - and none fabricated
The lab has no headless browser (Playwright/Chromium not installed), so visual (V3) evidence could not be produced without inventing it, which the governing methodology forbids. Genuine evidence is embedded instead: raw HTTP (Section 7), terminal output (Section 6), and authoritative DB reads (Section 8), all traceable to the sealed run runs/bpms/localhost/1793369337.

10 Impact

Dimension	Assessment

Dimension	Assessment
Confidentiality	Low/Moderate - disclosure of other customers' name, email, mobile, booking & invoice data
Integrity	None demonstrated
Availability	None

Realistic attack scenario

A registered customer sees that their own appointment/invoice numbers are 9-digit values and that the view pages accept any such number. With no rate limiting, they script requests across plausible ranges; each valid hit returns another customer's record - harvesting PII with no ownership relationship.

11 CVE Comparison & Novelty Assessment

Reference	Covers	Relationship to F-NEW-01
CVE-2024-51066	IDOR in appointment-detail.php (CWE-639)	Amplifier, NOT a duplicate - missing ownership vs. predictable id (CWE-330)
CVE-2024-53480 + 2025-26 SQLi set	SQL injection across BPMS	Unrelated class (CWE-89)
CVE-2021-27544; profile.php XSS	XSS in BPMS	Unrelated class (CWE-79)

A cross-check of NVD, MITRE, VulDB and cvefeed (2026-10-30) found no published CVE describing insecure randomness / predictable identifiers for this product; F-NEW-01 appears unreported. Honest caveat: because its practical impact overlaps the root concern of CVE-2024-51066 (weak object references), a CNA may treat it as related rather than wholly independent. No CVE is claimed or assigned.

12 Remediation

Primary - enforce ownership (root cause)

```
$stmt = $con->prepare(
  "select ... from tblinvoice
     join tbluser on tbluser.ID = tblinvoice.Userid
    where tblinvoice.BillingId = ? and tblinvoice.Userid = ?");
$stmt->bind_param("ii", $invid, $_SESSION['bpmsuid']);
```

Defence in depth - CSPRNG

```
// Before
$aptnumber = mt_rand(100000000, 999999999);
// After
$aptnumber = random_int(1000000000, 9999999999);   // or bin2hex(random_bytes(16))
```

Add rate limiting and generic "not found" responses on the lookup endpoints. An identifier that controls access is a capability: it must be unguessable AND must not be the only check. Fixing ownership first closes the issue even if a value leaks.

13 Verification Checklist

Check	Result
Source verified (file:line)	PASS - book-appointment.php:16; add-customer-services.php:12
Runtime verified	PASS - AptNumbers 593951775, 344823593; BillingId 193862954
PoC reproduced	PASS - non-destructive; DB restored to baseline
HTTP evidence captured	PASS - request.txt / response.txt (+ real headers)
Screenshots embedded	N/A - no browser in lab; stated honestly, not fabricated
CVE comparison completed	PASS - vs CVE-2024-51066 and SQLi/XSS clusters
Evidence sealed	PASS - runs/bpms/localhost/1793369337, SEAL 780fe797...

This report is a DRAFT for human review and manual coordinated disclosure to PHPGurukul. Nothing has been submitted to any third party.